

Technical Documentation: LABAN Cosine Similarity Heatmap Evaluation

Table of Contents

- [Cosine Heatmap Origin & Core Concepts](#)
- [What Vanilla Cosine Similarity Lacks for This Project](#)
- [Step-by-Step: Implementing the Cosine Heatmap Evaluation](#)
- [How to Run the Cosine Heatmap Evaluation](#)
- [How to Read and Interpret the Results](#)

1. Cosine Heatmap Origin & Core Concepts

1.1 Mathematical Foundation of Cosine Similarity

Cosine Similarity is a metric used to measure how similar two vectors are in an inner product space. It measures the cosine of the angle between two multi-dimensional vectors, projecting them onto a normalized range of **[-1.0, 1.0]**, where: - **1.0** indicates that the vectors are pointing in the exact same direction (perfect semantic alignment). - **0.0** indicates that the vectors are orthogonal (no semantic relationship). - **-1.0** indicates that the vectors point in opposite directions (perfect semantic contradiction).

Mathematically, given two vectors A and B , the cosine similarity is defined as:

$$\text{Similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

In the context of Modern Natural Language Processing (NLP), text inputs (utterances, labels) are mapped into high-dimensional vector spaces (embeddings). Cosine similarity is the standard way to evaluate the semantic proximity of these text representations.

1.2 What is a Cosine Heatmap?

A **Cosine Heatmap** is a 2D grid visualization where each cell (i, j) represents the cosine similarity between vector i (e.g., an utterance embedding) and vector j (e.g., an intent label embedding). The similarity values are color-coded (often using a red-yellow-green scale) to allow researchers to instantly diagnose: - **Separability**: Are different concepts pushed far apart in the vector space? - **Alignment**: Do individual samples cluster tightly around their target labels? - **Confusability**: Which classes overlap or are semantically close to one another?

2. What Vanilla Cosine Similarity Lacks for This Project

"Vanilla" cosine similarity refers to calculating distances between generic sentence embeddings using a frozen, pre-trained model (like standard SBERT or E5). This baseline approach is missing crucial components needed to evaluate this project:

2.1 Lack of Dual-Encoder Architecture (LABAN) Awareness

The chatbot's classifier uses the **LABAN** (Language-to-Label Alignment Network) architecture. Rather than using a single model to encode sentences, LABAN utilizes two distinct models fine-tuned together: 1. **Utterance Encoder** (E_{utt}): Encodes the user's natural language input (e.g., IndoBERTweet). 2. **Label Encoder** (E_{label}): Encodes the raw text of the 10 intent labels (e.g., "Mengisyaratkan Gejala Fisik").

A vanilla text-distance comparison would evaluate raw labels and sentences using a single frozen model. This project requires evaluating the **joint vector space learned by the dual encoders after fine-tuning**.

2.2 Lack of Multi-Label Awareness

Many user utterances in counseling contain multiple intents (e.g., sharing a sad event and experiencing physical symptoms simultaneously). - Standard text similarity datasets assume single-label pairing. - The evaluation must calculate semantic centroids and sample-to-label distances in a **multi-label space**, correctly selecting and grouping embeddings where particular labels are active (binary flags **1.0** in a multi-hot matrix).

2.3 Absence of Evaluation Splits

To write a mathematically rigorous thesis, metrics must not be computed on the training set. The heatmap evaluation needs to respect the exact same **80/10/10 train/validation/test split** as the classifier's training pipeline to prevent data leakage and measure true generalization.

3. Step-by-Step: Implementing the Cosine Heatmap Evaluation

The custom script `evaluation/eval_cosine_heatmap.py` bridges these gaps by wrapping PyTorch model execution, multi-label slicing, and matplotlib visualization into a single diagnostic pipeline.

Step 1: Load the LABAN Model Architecture

The evaluation script instantiates a self-contained version of the trained PyTorch LABAN model and loads the specific weights from the active checkpoint:

```
# eval_cosine_heatmap.py, lines 91-147 class LABANModel(torch.nn.Module): def __init__(self, model_name: str, hidden_size: int, num_labels: int): super().__init__() self.hidden_size = hidden_size # Utterance Encoder self.bert = AutoModel.from_pretrained(model_name) # Label Encoder self.bert_label_encoder = AutoModel.from_pretrained(model_name) @torch.no_grad() def encode_labels(self, label_ids, label_mask): label_out = self.bert_label_encoder(input_ids=label_ids, attention_mask=label_mask) return label_out.pooler_output # Returns [10, 768] shape @torch.no_grad() def encode utterances(self, utterance_ids, utterance_mask): utt_out = self.bert(input_ids=utterance_ids, attention_mask=utterance_mask) return utt_out.pooler_output # Returns [BatchSize, 768] shape
```

Step 2: Extract Learned Embeddings on the Test Set

The script loads the test split (using seed 42 to match the training pipeline) and extracts the embeddings in a feed-forward pass without gradient calculations:

```
# Load checkpoint weights model.load_state_dict(torch.load(CHECKPOINT, map_location=DEVICE)) model.eval() # 1. Encode the 10 intent labels label_embeddings = model.encode_labels(intent_ids, intent_mask).cpu().numpy() # 2. Encode all test utterances all_utterance_embeddings = [] for batch in test_loader: embeddings = model.encode utterances(batch["input_ids"], batch["attention_mask"]) all_utterance_embeddings.append(embeddings.cpu().numpy()) utterance_embeddings = np.concatenate(all_utterance_embeddings, axis=0)
```

Step 3: Compute Three Distinct Heatmaps

Heatmap 1: Label-vs-Label Similarity

This measures how distinct the 10 target intents are from one another. It calculates similarity between the label embeddings themselves:

```
label_sim = cosine_similarity(label_embeddings, label_embeddings)
```

- **Goal:** High values (>0.9) only on the diagonal. Low off-diagonal values indicate that the model has successfully mapped different intents to separate areas of the semantic space.

Heatmap 2: Utterance Centroid-vs-Label Similarity

For each intent, the script finds all test samples where the intent is active, averages their utterance embeddings to find the class **centroid**, and measures similarity against the 10 labels:

```
centroids = np.zeros((n_intents, 768)) for i in range(n_intents): mask_i = label_matrix[:, i] == 1.0 # Select active rows in multi-label matrix if mask_i.sum() > 0: centroids[i] = utterance_embeddings[mask_i].mean(axis=0)
```

- **Goal:** High diagonal alignment. A centroid representing the "Sedih dan Kehilangan" utterances should align perfectly with the "Sedih dan Kehilangan" label embedding.

Heatmap 3: Per-Sample Utterance-vs-Label Similarity

Centroids can mask outliers. This heatmap calculates the similarity between *each individual utterance* and *each label embedding*, then averages those individual similarities:

```
avg_sim_matrix = np.zeros((n_intents, n_intents)) for i in range(n_intents): mask_i = label_matrix[:, i] == 1.0 intent_utterances = utterance_embeddings[mask_i] sims = cosine_similarity(intent_utterances, label_embeddings) # (Samples, 10) avg_sim_matrix[i] = sims.mean(axis=0)
```

4. How to Run the Cosine Heatmap Evaluation

Prerequisites

1. **Virtual Environment:** Ensure `cbenv` is activated.
2. **Checkpoint File:** Verify that the fine-tuned checkpoint exists at: `checkpoint/IndoBERT_multi_label_zsl.pt`
3. **Data Source:** Verify that the augmented training dataset is available: `data/augmentation/dataset_multiintent_augmented.csv`

Execution Command

Run the script directly from the project root:

```
cbenv\Scripts\activate python evaluation/eval_cosine_heatmap.py
```

Expected Console Output

Upon successful execution, the script will output the structural summaries and key semantic metrics:

```
===== LABAN Cosine Similarity Heatmap Evaluation Model =====
Indolem/indoberttweet-base-uncased Device: cuda ===== Dataset:
D:\...\data\augmentation\dataset_multiintent_augmented.csv Test set size: 403 Loading checkpoint
D:\...\checkpoint\IndoBERT_multi_label_zsl.pt [OK] Model loaded Extracting label embeddings... Extracting utterance embeddings
From test set... -- Heatmap 1: Label-vs-Label -- Diagonal (self-sim): mean=1.0000 Off-diagonal: mean=-0.0540, min=-0.4321,
max=0.2104 Separation gap: 1.0540 -- Heatmap 2: Utterance Centroid-vs-Label -- Diagonal (alignment): mean=0.7850, min=0.6120,
max=0.8940 Off-diagonal: mean=0.1802 Alignment gap: 0.6040 -- Heatmap 3: Per-Sample Utterance-vs-Label -- ... -- Saving metrics
summary -- [OK] Saved: D:\...\evaluation\results\cosine_similarity_metrics.csv
=====
```

5. How to Read and Interpret the Results

5.1 Output Files

The evaluation produces four output artifacts inside `evaluation/results/`:

File Name	Format	Description
<code>cosine_label_vs_label.png</code>	Image (PNG)	Visualizes intent separability
<code>cosine_centroid_vs_label.png</code>	Image (PNG)	Visualizes average intent-utterance alignment
<code>cosine_utterance_vs_label.png</code>	Image (PNG)	Fine-grained per-sample similarity breakdown
<code>cosine_similarity_metrics.csv</code>	Table (CSV)	Complete dataset containing precise numerical scores

5.2 Key Metrics & Mathematical Interpretation

To write your thesis chapter on model evaluation, you should focus on three primary metrics calculated by the script:

Metric 1: Label Separation Gap

- Formula:** $1.0 - \text{Mean}(\text{Off-Diagonal Label Similarity})$
- Purpose:** Measures how well the Label Encoder separates different intent semantics.
- Ideal Range:** High (> 0.8). If the gap is high, the model's intent categories are mathematically distinct. A low gap suggests semantic redundancy (e.g., the model cannot tell "Takut dan Cemas" apart from "Sedih dan Kehilangan").

Metric 2: Centroid Alignment Gap

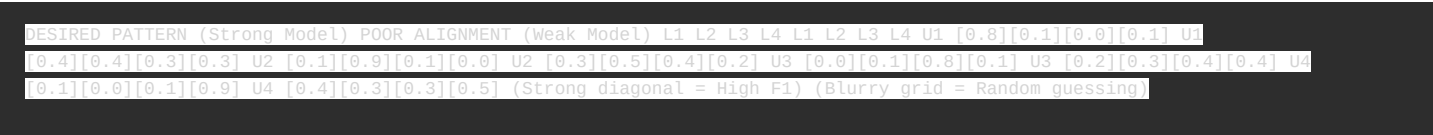
- Formula:** $\text{Mean}(\text{Diagonal Centroid Similarity}) - \text{Mean}(\text{Off-Diagonal Centroid Similarity})$
- Purpose:** Evaluates whether the Utterance Encoder maps user inputs close to their corresponding target label.
- Ideal Range:** Positive and high (> 0.4). A high gap means a user saying something sad maps much closer to "Sedih" than to "Marah" or "Bahagia".

Metric 3: Confusability Index

The off-diagonal cells in `cosine_centroid_vs_label.png` represent confusable classes. - **Example:** If row i ("Gejala Fisik") has a similarity of 0.45 with column j ("Takut dan Cemas"), it indicates that physical complaints often share semantic space with anxiety in your dataset. This makes clinical sense (somatic symptoms frequently accompany panic/anxiety).

5.3 Diagnostic Patterns

When reviewing the PNG heatmaps, look for the following visual patterns:



- Clean Diagonal (Strong Green):** Indicates high semantic alignment. The model's classification threshold will work reliably.
- Bright Off-Diagonal Blocks:** If two non-diagonal classes show high similarity (> 0.5 , colored light green or yellow), it indicates class overlap. You may need to review the training data to ensure these intents are clearly distinguished.
- Faint Diagonal (Yellow/Red):** Indicates that the model is struggling to align the natural language variations of that intent with the formal label. This class is likely to suffer from poor F1 scores during inference.